AI-SOC: Person B Plan (Intelligence, Security and Experience)

Standalone plan for the person who owns the LLM investigation, login and approvals, the user interface and the evaluation · Dates match the main plan: 2 Nov 2026 to 25 Apr 2027 · Plan date: 28 Oct 2026

1. Mission and boundaries

Person B makes the system's reasoning trustworthy, visible and measurable. The model's output is validated and never trusted; every human decision is authenticated and recorded; every claim in the final report is backed by a number from the evaluation. B's results chapter is the project's headline evidence.

B owns: the investigation package (Ollama client, evidence pack, prompt, schema, validators, fallback), the security and approval packages (login, roles, API key check, approval workflow), the entire React frontend, the evaluation dataset and harness, the prompt-injection test set, the ATT&CK lookup, Docker Compose, and the report chapters listed in section 8.

B does not own: the Windows monitor, ingestion, detection rules, policy engine, audit log, state machine and remediation executor. Those belong to Person A. B consumes them through the agreed interfaces and never edits A's packages without a reviewed pull request.

2. Deliverables and acceptance criteria

D1. Investigation service. OllamaInvestigationService behind the existing interface, switchable by config between mock, ollama and ollama-with-fallback.

- Accepts: an incident evidence object; returns a stored, versioned investigation
- Runs asynchronously with a status (PENDING, RUNNING, COMPLETED, FALLBACK, FAILED); never blocks the API
- Done when: 20 consecutive investigations on fixtures finish with a valid stored result and no unhandled exception

D2. Validation pipeline. Schema, grounding and consistency checks with retry and a rule-based fallback.

- Done when: six simulated failures (malformed JSON, wrong enum, invented IP, wrong count, timeout, connection refused) each end in a stored FALLBACK result, never an exception, and never an action

D3. Security and approvals. Login, roles, monitor API key check, approval workflow.

- Done when: the authorization matrix (every role against every endpoint) passes, a forged alert without a key is rejected, and an approval can never be applied to parameters other than the ones shown to the approver

D4. Frontend. Login, incident list, incident detail with investigation panel, response plan and policy decision panel, approval inbox, action status timeline, audit view, and (Should) analytics.

- Done when: the full demo flow runs from the browser with no console errors and every screen has loading, empty and error states

D5. Evaluation. 80 to 100 labelled scenarios, a repeatable harness, four or five configurations, result tables and charts.

- Done when: one command reproduces every table in the report from the stored scenario files

D6. Red-team set. 20 to 30 prompt-injection and adversarial cases with documented results and fixes.

- Done when: no case changes a policy decision, produces an action outside the allowed list, or escapes the schema

D7. Packaging. Full Docker Compose for Postgres, backend, frontend and optionally Ollama, with a README that starts the system from a clean machine.

D8. Writing and demo. Report chapters, the investigation and approval half of the demo, viva preparation.

3. Design details

3.1 Investigation pipeline

1. **Evidence pack:** from the incident and its alerts, compute the facts in code: number of failures, time span, distinct source IPs, distinct accounts, logon types, whether a success followed failures, rule that fired. The model is given these facts and never has to count.

2. **Prompt:** a versioned text file in the repository (investigation-v1, v2, and so on). It states the role, says that everything in the evidence block is data and not instructions, lists numbered evidence items, and describes the output.
3. **Call:** Ollama chat endpoint without streaming, temperature 0, a fixed seed, a limited context size, keep-alive so the model stays loaded, a JSON schema in the format field, and a timeout. Warm the model with one call at startup.
4. **Validate:** section 3.3.
5. **Retry once** with the validation error appended, then fall back.
6. **Store:** raw response, parsed result, validation report, model name and digest, prompt version, parameters, latency, token counts, and the source label (LLM, FALLBACK or MOCK). Keep every run; never overwrite.

Structured output forces valid JSON but does not make it true. That is why grounding checks exist.

3.2 Output contract (what the model may produce)

- classification: one value from a fixed list (brute force, password spraying, compromise suspected, benign user error, misconfiguration, insufficient evidence)
- summary: short text with a length cap
- findings: statements, each citing evidence item numbers
- hypotheses: statements clearly marked as unverified, each with what would confirm them
- recommended action type: one value from a fixed list (none, monitor, notify, block source IP, disable account)
- confidence: number from 0 to 1
- uncertainties: short list

Not produced by the model: severity, ATT&CK technique ID and approval requirement (set by rules and A's policy engine), and the action target. The model picks an action type only; code fills the IP or account from the incident evidence, so the model never types an address.

3.3 Validators

1. Schema, enums, lengths and ranges
2. Every cited evidence number exists in the pack

3. Entity grounding: every IP, username, host and number in the text appears in the evidence pack or computed facts
4. Count consistency: any quantity the text states equals the computed fact
5. Forbidden claims: threat-intelligence, attribution or CVE statements are rejected unless provided as evidence
6. Action compatible with classification (for example, no block action for benign user error)
7. Technique, if mentioned, equals the rule's technique and is in the ATT&CK allow-list
8. Confidence clamped; low confidence marks the result for analyst review

Every check writes a pass or fail line to the validation report so the UI can show exactly what was rejected and why.

3.4 Data and asynchronous design

- Investigations table: incident id, version, source, status, model, model digest, prompt version, parameters, raw response, parsed result, validation report, latency, tokens, created time
- A background executor runs jobs; the UI polls status (or uses server-sent events) and shows progress
- Configuration: mode, model name, timeout, retries, confidence threshold, all in properties with environment overrides
- Migrations are Flyway files with date-based version numbers, coordinated with Person A to avoid clashes

3.5 Security and approvals

- **Sessions:** Spring Security with an HttpOnly, SameSite session cookie and CSRF protection. In development use the Vite proxy; in Compose use one reverse proxy, so the browser sees a single origin and no tokens live in JavaScript.
- **Users and roles:** ANALYST (view, feedback), APPROVER (approve, reject), ADMIN (policy and configuration). BCrypt password hashes; seed accounts created from environment variables; no default passwords in the repository; login rate limiting.
- **Monitor API key:** random 32-byte key per host, stored only as a hash, sent in a header, compared in constant time, revocable. Missing or bad key means 401 and an audit event.

- **Approval request:** created when A's policy engine returns REQUIRE_APPROVAL. It stores the exact action parameters and a hash of them. The approver sees those parameters; approval is valid only for that hash. Requests expire after a configured time (for example 30 minutes) and expired requests cannot be approved.
- **Decisions:** approve or reject (reject needs a reason); each writes an audit event. Optional (Should): re-enter password to approve.
- **Hardening:** CORS locked to the known origin, request validation, size limits, secrets only from the environment.

3.6 Frontend

- **Screens:** Login; incident list (filters by severity, status, rule); incident detail (alerts, investigation, source badge, validation warnings, evidence shown separately from hypotheses); response plan and policy decision with reason codes; approval inbox; action timeline (proposed, checked, approved, executing, verified, rolled back); audit log (filter, search); analytics (Should).
- **Rules:** types generated from the OpenAPI file; every screen has loading, empty and error states; destructive buttons need confirmation; the UI never shows an approve button to a role that cannot use it, and the server enforces it anyway.
- **Tests:** component tests for the investigation panel and approval inbox; browser tests for the main flow and for a rejected approval.

3.7 Evaluation method

- **Dataset:** JSON scenario files, each with the event set, ground-truth classification, expected action type and a note. Categories: single-account brute force, password spraying, brute force then success, benign typos, lockout after a password change, misconfigured service account, low-and-slow attacker, two attackers at once, plus adversarial cases (hostile username, empty evidence, conflicting evidence).
- **Splits:** 60 percent development, 40 percent held-out test. Tune the prompt on development only; freeze the prompt version before touching the test set.
- **Labelling quality:** Person A and Person B both label a sample of 30 scenarios independently; report agreement (Cohen's kappa) and resolve differences.
- **Configurations:** A rule-only or mock; B raw LLM output without validation; C LLM with schema only; D full system; E full system with a larger model if hardware allows.
- **Runs:** three runs per LLM configuration at temperature 0 with a fixed seed; report mean and range.

- **Metrics:** schema-valid rate, classification accuracy and macro-F1, action-type accuracy, unsupported-claim rate (automatic entity check plus manual review of a sample), severity and risk consistency, fallback rate, p50 and p95 latency. Person A supplies the detection, policy and remediation metrics.
- **Harness:** a script (Python with pandas and matplotlib is convenient) that replays scenarios into the backend, records every result and produces the tables and charts. Results are stored with the prompt version and model digest so they can be reproduced.

3.8 Prompt-injection and adversarial set

Cases to include: instructions hidden in a username or workstation name, fake evidence markers, very long fields, control characters, text claiming to be a system message, requests to recommend blocking the gateway, requests to ignore the schema, and mixed-language text. Defences: delimiters and a data-not-instructions statement, length caps, control-character stripping, schema-constrained output, entity grounding, and the policy engine ignoring anything not on the allowed list. Document each case, the result before and after fixes.

4. Week-by-week plan

Phase 0 and 1: Foundations (2 to 22 Nov)

- **Week 1 (2 to 8 Nov):** audit the frontend and investigation code and note stub versus real; add the frontend build to CI; agree the six interfaces with Person A and publish the OpenAPI file; define the investigation schema (B's contract); draft the classification taxonomy; benchmark qwen2.5:3b on your machine with a realistic evidence pack (latency, memory) and record the numbers
- **Week 2 (9 to 15 Nov):** build 12 incident fixtures and a development profile that loads them; set the scenario file format and write the first 15 scenarios; clean up the dashboard (filters, detail layout, loading, empty and error states)
- **Week 3 (16 to 22 Nov):** prototype prompt and schema against Ollama on the fixtures with a standalone script; grow the dataset to 30 labelled scenarios; write the validator design; add OpenAPI stubs for investigation and approval endpoints
- **Gate:** fixtures and 30 scenarios exist; prompt prototype gives schema-valid output on most fixtures

Phase 2: Ollama integration (23 Nov to 13 Dec)

- **Week 4 (23 to 29 Nov):** OllamaInvestigationService (client, config, timeouts, warm-up); evidence pack builder (on fixtures until Person A's evidence object lands); background job and status; investigations table migration
- **Week 5 (30 Nov to 6 Dec):** validators 1 to 8; retry then fallback; rebuild the mock as a rule-based fallback from real incident data; stubbed-Ollama tests for the six failure modes
- **Week 6 (7 to 13 Dec):** UI source badge, validation warnings, evidence versus hypotheses; harness v1; run 30 scenarios; first accuracy table; define the development and test split
- **Gate:** every failure mode ends in a safe fallback; first accuracy table exists

Stage I (14 to 20 Dec): write the literature survey and investigation design chapters; prepare slides and the investigation half of the demo; fix bugs from review.

Buffer (21 Dec to 3 Jan): tests and writing only. Review Person A's pull requests. Draft the literature survey in full.

Phase 3: Security groundwork (4 to 17 Jan)

- **Week 10 (4 to 10 Jan):** user and role model, BCrypt, session configuration, seeded accounts from environment, login page and logout
- **Week 11 (11 to 17 Jan):** write audit events from investigation and login using Person A's audit format; build the response plan and policy decision panel against A's format
- **Gate:** login works end to end; investigation events appear in the audit log

Phase 4: Approvals (18 to 31 Jan)

- **Week 12 (18 to 24 Jan):** monitor API key table, filter and rotation; role checks on every endpoint; approval domain (request, parameter hash, expiry job); approval endpoints
- **Week 13 (25 to 31 Jan):** approval inbox and audit view; CSRF and CORS configuration; login rate limiting; authorization test matrix
- **Gate:** an unapproved action never reaches the executor; forged alert rejected; approval bound to the exact parameters

Phase 5: Integration with remediation (1 to 21 Feb)

- **Week 14 (1 to 7 Feb):** action status timeline from Person A's state machine; wire approval to execution
- **Week 15 (8 to 14 Feb):** failure-state screens (executor offline, verification failed, rolled back, expired approval); browser tests for the main flow

- **Week 16 (15 to 21 Feb):** seeded replay script so the demo is reproducible; rehearse the investigation and approval half
- **Gate:** scripted attack leads to an approved, verified block that is removed on expiry, visible in the UI

Phase 6: Evidence and hardening (22 Feb to 7 Mar)

- **Week 17 (22 to 28 Feb):** evidence provenance (each finding links to its evidence in the UI); incident timeline; ATT&CK lookup file and technique validation
- **Week 18 (1 to 7 Mar):** prompt-injection set of 20 to 30 cases; fixes; documented before and after results
- **Gate:** no injection case bypasses policy or escapes the schema

Phase 7: Evaluation and packaging (8 to 21 Mar)

- **Week 19 (8 to 14 Mar):** expand to 80 to 100 scenarios including adversarial cases; cross-label 30 with Person A and compute agreement; freeze the prompt; run configurations A to D three times each
- **Week 20 (15 to 21 Mar):** model comparison (E) if hardware allows; latency statistics; tables and charts; analytics dashboard only if on schedule; full Docker Compose and a clean-machine start test
- **FEATURE FREEZE 21 Mar**

Phase 8: Writing and viva (22 Mar to 25 Apr)

- **Weeks 21 and 22 (22 Mar to 4 Apr):** results and discussion chapters; evaluation appendix; demo video script
- **Weeks 23 and 24 (5 to 18 Apr):** full report read-through with Person A; slides; record the demo; viva question rehearsal
- **Week 25 (19 to 25 Apr):** buffer, final fixes, tag v1.0

5. What B needs from Person A, and when

- **By 8 Nov:** alert schema, incident evidence object, audit event format, policy decision and response plan format, action state enum (until then, B uses fixtures)
- **By 22 Nov:** Detection v2 producing real incidents (B swaps fixtures for real data)
- **By 17 Jan:** policy engine returning ALLOW, REQUIRE_APPROVAL or DENY with reason codes (B builds approval requests from it)

- **By 31 Jan:** executor accepting an approved action (until then A uses a test-only approved action)
- **By 1 Feb:** state machine events and status endpoint for the timeline

If any dependency is late, stub it behind the agreed interface and keep going; raise it at the Friday integration session.

6. Test plan for B's work

- **Unit tests:** each validator with passing and failing examples; evidence pack facts; approval expiry and hash binding
- **Contract tests:** stubbed Ollama (WireMock) for valid, malformed, wrong-enum, invented-entity, wrong-count, timeout and refused-connection responses
- **Integration tests:** Testcontainers with PostgreSQL for investigation storage and approval flow
- **Security tests:** role-by-endpoint matrix; missing and bad API key; expired and tampered approval; CSRF; login rate limit
- **Frontend tests:** component tests and browser tests for login, investigation view, approval, rejection
- **Chaos checks:** stop Ollama mid-run; stop the database; submit the same alert twice
- **CI:** all of the above run on every push; merges require a green build

7. Risks specific to B and mitigations

- **Slow inference on your hardware:** asynchronous jobs, strict timeouts set from measured p95, keep-alive, caching of identical evidence packs
- **Model quality too low:** deterministic fields, validators and fallback are the design; if quality is poor that is a measured result. Try a larger model for configuration E if memory allows
- **Overfitting the prompt to your own scenarios:** development and test split, prompt frozen before the test run, held-out adversarial cases
- **Labelling bias:** cross-label with Person A and report agreement
- **Interface drift:** contracts in OpenAPI and shared schemas; changes only by reviewed pull request
- **UI scope creep:** analytics and extras are Should items; the cut line comes from the main plan

- **Approval bypass bugs:** server-side enforcement, tests for every role, hash-bound approvals
- **Exams and holidays:** buffer weeks are used for tests and writing, not features

8. Report chapters and viva

Chapters B writes: literature survey (SOC automation and SOAR, LLMs in security operations, ATT&CK, prompt injection, human-in-the-loop control); investigation design; authentication and approvals; evaluation method; evaluation results for investigation; the discussion of model failures.

Writing habit: write each section within a week of finishing the matching feature, while details are fresh. Keep a running log of prompt versions and what each change did.

Viva questions B must answer well:

- Why a local model, and why a 3B one?
- Why does the LLM not decide severity, technique or approval?
- How do you stop prompt injection through log fields?
- What happens when Ollama is down or returns garbage?
- How exactly did you measure accuracy and unsupported claims, and how do you know the labels are right?
- What stops a forged alert, or a tampered approval, from causing a block?
- How does this differ from Wazuh active response or a commercial SOAR product?

B should also understand Person A's half well enough to explain the detection rules and the policy engine; schedule the 15-minute teach-back after each phase.

9. First 14 days checklist

1. Read the audit of the frontend and investigation code; list real versus stub
2. Add the frontend build to CI
3. Agree the six interfaces with Person A; publish the OpenAPI file
4. Write the investigation JSON schema and the taxonomy
5. Benchmark qwen2.5:3b on a realistic evidence pack and record latency and memory
6. Build 12 fixtures and the development profile
7. Write the first 15 scenarios with labels

8. Clean up the dashboard states (loading, empty, error)
9. Prototype the prompt on the fixtures and save every version
10. Set up the contribution log

10. Definition of done for Person B

- All eight deliverables in section 2 meet their acceptance criteria
- Every table and chart in the report regenerates from stored data with one command
- The investigation and approval half of the demo runs from a clean start without edits
- The full report has been read by Person A and the viva questions in section 8 have been rehearsed aloud